IOT BASED AD-HOC SYSTEM FOR SAVING

Real-Time Driver and Road Monitoring for Risk Prediction

25-26J-291

Project Proposal Report

Samoda T.W.O

B.Sc. (Hons) Degree in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

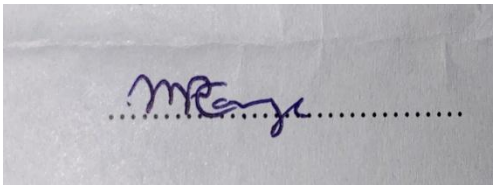
August 2025

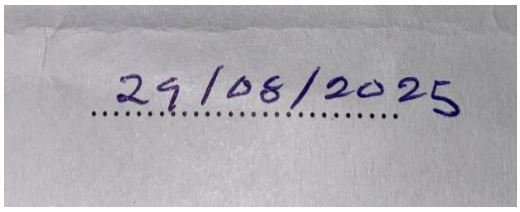
Declaration of The Candidate & Supervisor

I declare that this is my work, and this proposal does not incorporate without acknowledgment any material previously submitted for a degree or diploma in any other university or institute of higher learning, and to the best of our knowledge and belief, it does not contain any material previously published or written by another person except where the acknowledgment is made in the text.

Name	Student ID	Signature
Samoda T.W.O	IT22257468	

The above candidates are carrying out research for the undergraduate Dissertation under my supervision.





....

Signature of the supervisor Date

Acknowledgment

This paper and the research behind it would not have been possible without the exceptional support of my supervisors, my Research supervisor Mrs. Manori Gamage Co-supervisor Mr. Nelum Chathuranga Their enthusiasm, knowledge, and exacting attention to detail have been an inspiration and kept my work on track. Besides my supervisors, I would like to thank the rest of my group members Senadheera S.C.C, Pinsara T.H.A.K, D.L.R Dilochana for their encouragement, insightful comments, and every other support. And finally, thanks to my parents who always offer support and love.

Abstract

This research aims to address the critical need for a robust real-time driver assistance and road monitoring system to mitigate accident risks through the integration of computer vision and IoT connectivity. A key focus is the rapid detection of road signs (especially speed limits) and adverse conditions (such as bad weather), and the immediate transmission of warnings to drivers to encourage corrective action. The proposed solution employs onboard cameras for continuous road sign monitoring and speed limit recognition, alongside sensors and online data to monitor environmental conditions, with a real-time alert. The innovative aspect of this study lies in the fusion of traffic sign recognition technology with dynamic risk evaluation based on vehicle speed and weather conditions, creating a proactive safety mechanism rather than a reactive one. This approach reinforces traffic safety protocols and facilitates prompt, well-informed driver responses to potential hazards. By l everaging cutting-edge technologies – including image processing, IoT sensors, and real-time data analytics – the system is poised to transform road safety management. Through accurate detection of driving context and instantaneous communication of risk alerts, stakeholders (drivers and traffic management centers) can respond swiftly to emerging threats, implementing timely interventions to prevent accidents. Furthermore, this integration of advanced technologies marks a paradigm shift toward preventative road safety strategies that prioritize accident avoidance over post-accident reaction. Fundamentally, this research represents a significant step toward reducing road traffic collisions and fatalities, highlighting the indispensable role of intelligent technological systems in contemporary transportation safety efforts.

Keywords: Driver assistance, Traffic sign recognition, IoT, Weather conditions, Real-time alerts, Road safety.

Table of Contents

Table of Contents

Declaration Of The Supervisor..... 2

Acknowledgement..... 3

Abstract 4

Table of Contents 5

[1] INTRODUCTION..... 6

1.1. Background & Literature Survey..... 6

1.2. Literature Survey..... 7

1.3. Research Gap 9

1.4. Research Problem..... 11

[2] OBJECTIVES..... 13

2.1. Main Objective..... 13

2.2. Specific Objectives..... 15

[3] METHODOLOGY..... 17

3.1. Project Overview..... 17

3.2. System Over View Diagram..... 20

3.3. Individual Component..... 23

3.3.2. The Flow of the System – Individual Component 23

3.4. Tools And Technologies..... 26

[4] PROJECT REQUIREMENT..... 33

[5] WORK REAKDOWN CHART..... 36

[6] GANTT CHART..... 37

[7] BUDGET AND BUDGET JUSTIFICATION..... 39

[8] REFERENCES..... 41

1. INTRODUCTION

1.1. Background & Literature Survey

Road traffic accidents are a leading cause of death and injury worldwide, imposing a grave public health and economic burden. Every year, approximately 1.32 million people are killed in road crashes globally, with tens of millions more injured pmc.ncbi.nlm.nih.gov. Low- and middle-income countries bear a disproportionate share of these fatalities, reflecting challenges in traffic safety management pmc.ncbi.nlm.nih.gov. In Sri Lanka, as in many countries, road accidents have become a critical concern – averaging around 3,000 deaths annually in recent years asiantransportobservatory.org. Speeding and driver error are consistently identified among the top contributors to these accidents. For instance, a comprehensive local study of Sri Lankan road crashes (2002–2019) found that **"speed driving" was a contributing factor in about 13.3%** of accidents, alongside other risky behaviors like overtaking (16.5%)epid.gov.lk. This aligns with global findings that excessive speed is one of the largest behavioral contributors to road fatalities researchgate.net. In fact, research indicates that **every 1% increase in a vehicle's average speed results in roughly a 4% increase in the fatal crash risk**, underlining how critical effective speed management is for road safety pmc.ncbi.nlm.nih.gov.

Another major factor influencing road accidents is adverse weather. Rain, fog, and other poor weather conditions can drastically reduce visibility and road surface friction, leading to longer braking distances and loss of vehicle control. Studies show that a significant percentage of crashes are weather-related – for example, about 12% of annual crashes in the United States occur under adverse weather conditions, with the vast majority of those happening on wet roads during rainfall ops.fhwa.dot.gov. Heavy rainfall in particular is known to increase accident risk by causing water buildup (hydroplaning) and impairing driver vision. Sri Lanka, being a tropical country with seasonal heavy rains, experiences these hazards regularly. Local research by Pushpakumara et al. (2012) confirmed a **strong correlation between rainfall and road accident rates** in the Colombo district, emphasizing that weather conditions "greatly affect road crashes" by altering road and driver conditions researchgate.net. Despite this clear link, conventional road safety measures in Sri Lanka (and many other regions) do not dynamically adjust to weather; speed limits remain fixed, and drivers rely on personal judgment, which can be flawed, especially at night or in sudden storms.

Traditionally, efforts to reduce accidents from speeding and bad weather have relied on static and after-the-fact measures. **Static road signs** (e.g., speed limit signs, warning signs) and **manual enforcement** (traffic police patrols and speed traps) have been the mainstay of speed management. While these measures are important, they have limitations. Fixed signs cannot account for dynamic conditions – a safe speed on a dry, clear day might be unsafe during a downpour, yet the posted limit stays the same. Police enforcement, on the other hand, is sporadic and often catches violations only after they occur rather than preventing them. There is often a delayed or insufficient response to hazards on the road (like an accident ahead or sudden fog), meaning drivers are not alerted in time. Figure 1 summarizes some of the key challenges in relying solely on traditional methods for real-time driver safety.

1.2. Literature Survey

A broad review of literature highlights the multi-dimensional nature of the road safety problem and the emerging technological solutions aimed at addressing it. On the one hand, numerous studies dissect the **causes and human factors** behind road accidents. Many works in Sri Lanka and globally concur on the major risk factors: speeding, reckless driving maneuvers, and driver impairment are frequently at the top of the list pmc.ncbi.nlm.nih.govepid.gov.lk. These studies underscore that any effective solution must influence driver behavior, particularly with respect to speed control. Public health research often emphasizes the need for better enforcement and driver education, noting that community attitudes sometimes downplay the risks of "minor" speeding (e.g., going 5-10 km/h over the limit) despite evidence that even low-level speeding contributes significantly to crash tolls researchgate.net. Another thread of literature explores how **environmental factors** like weather and time of day contribute to crashes. Research in this domain uses statistical analysis to demonstrate, for example, that accident frequencies tend to spike during heavy rainfall or low-visibility conditions researchgate.netops.fhwa.dot.gov. Such findings bolster the argument that dynamic risk prediction (accounting for weather) should be integrated into road safety strategies, rather than relying solely on static rules.

On the other hand, engineering and computer science literature offers insight into **technological interventions** for improving road safety. A significant area of development is ADAS and intelligent transportation systems. Traffic Sign Recognition (TSR) systems, often using camera-

based computer vision, have matured to a point where they can reliably identify common road signs (such as speed limits) in real time. Researchers have applied deep learning models to achieve high accuracy in classifying speed limit signs under various conditions ijrpr.com. This technology has been implemented in some vehicles to notify drivers of the current speed limit or even automatically limit the vehicle's speed to comply with the detected sign. Another technology, Intelligent Speed Adaptation (ISA), actively helps drivers stay within speed limits; trials in Europe and Australia have shown ISA can reduce speeding instances significantly by either warning the driver or gently governing the engine to prevent excessive speed.

Beyond on-vehicle systems, the concept of **connected vehicles and infrastructure (V2X communication)** is gaining momentum. IoT-based approaches envision a network where vehicles, roadside units, and central traffic control communicate continuously. Studies by Zanella et al. (2014) and Al-Sakran (2015) demonstrate IoT's potential in creating "smart road" infrastructure that can broadcast safety data to drivers in real time ijraset.com. For instance, a smart roadside unit might combine inputs from weather sensors, traffic flow detectors, and accident reports to dynamically update digital speed limit signs or send alerts to approaching cars. Campolo et al. (2017) and Gozalvez (2016) discuss how dedicated short-range communications (DSRC) and cellular V2X technologies can be used to ensure these messages reach vehicles with low latency ijraset.com. These capabilities enable scenarios such as warning a driver that the vehicle ahead has performed an emergency brake or that there is a hazard around a blind curve – situations that traditional signage or human reaction alone could not adequately address. Meanwhile, data analytics and machine learning are being applied to **predictive safety**: by analyzing patterns in traffic data, algorithms can forecast high-risk situations (e.g., likelihood of a crash at a particular location and time) and preemptively alert drivers or adjust control systems ijraset.com.

Amid these advancements, several **research gaps** and challenges have been identified, which set the stage for the present study. One notable gap is the **integration of disparate functions** – many existing solutions focus on one aspect (for example, just speed enforcement or just weather warnings) in isolation. There is a lack of comprehensive systems that holistically combine sign recognition, vehicle dynamics, and weather condition monitoring into a single, synchronized driver alert platform. Additionally, the literature indicates that while prototype systems and simulations exist, real-world implementation (especially in developing countries) lags behind. Technologies proven in laboratories or specialized testbeds (like V2X communication) are not yet widely deployed on actual roads. Issues such as sensor cost, networking infrastructure, and user acceptance pose hurdles. Moreover, ensuring the **reliability** of these systems under all conditions is a challenge – e.g., computer vision can struggle with sign detection at night or in heavy rain, and communication networks may have latency or coverage gaps. Cybersecurity is another concern raised by researchers like Roman et al. (2018), given that an IoT-based road safety system would transmit sensitive data and could be a target for malicious interference ijraset.com.

In summary, the literature survey reveals that improving road safety, particularly in terms of speed and real-time hazard response, requires an interdisciplinary approach. Behavioral studies

highlight *what* problems to target (speeding, weather-related risk), while technological studies offer insight into *how* we might address them (through IoT connectivity, automation, and intelligent algorithms). By synthesizing these insights, this research aims to build upon prior knowledge and push the frontier towards a practical, integrated solution. The goal is a system that not only detects and warns but does so in a way that is rapid, reliable, and adaptable to conditions – thereby fostering safer driving behavior and reducing accidents.

1.3. Research Gap

Our research is driven by a clear **gap in current road safety systems**: the absence of an integrated, real-time solution that combines on-board sign detection, environmental awareness, and instant driver feedback. Existing measures and technologies, while valuable, each leave certain needs unmet when used in isolation. The table below contrasts key challenges in real-time driver and road monitoring with what has (or has not) been addressed by prior systems:

Table 1: Comparison of existing solutions and the challenges they address

Challenge (in real-time driver/road monitoring)	Static Signs & Traditional Enforcement	Vehicle ADAS (Traffic Sign Recognition, etc.)	IoT-Based Speed Enforcement (William & Olayemi 2025) [1]	Proposed System (2025)
Real-time on-board detection of speed limits	No – Relies on driver to notice signs	Yes – Camera can identify posted limits	Yes – Uses cameras/sensors to detect speed violations	**Yes** – Camera actively reads signs
Dynamic weather condition awareness (adapting to rain, fog, etc.)	No – Static; no adjustment for weather	No – Most do not incorporate weather factors	Yes – Can utilize weather data in speed optimization	**Yes** – Integrates rain sensors/API for weather
Immediate in-vehicle warnings to driver (preventive alerts vs. punitive action)	Limited – Warning signs only (no dynamic alert)	Yes – Some systems beep/display if overseeding	Partial – System issues alerts/fines (enforcement focus)	**Yes** – Audible/visual alerts to driver in real time

Adaptability to local context and low-cost deployment in developing regions	No – One-size-fits-all static approach	Partial – Only in vehicles that can afford ADAS	Partial – Concept is general, but implementation not region-specific	**Yes** – Tailored to local roads; uses affordable components

In this table:

- "Yes" indicates the solution addresses the challenge,
- "NO" indicates it does not, and "Partial" indicates it addresses the challenge to some extent or indirectly.

As shown in Table 1, **traditional methods** (static signage and police enforcement) do not provide real-time adaptive support to drivers – they neither adjust to weather nor give immediate personalized feedback in most cases. Modern **ADAS in vehicles**, like built-in sign recognition, do better by detecting signs and sometimes warning the driver, but they typically lack the ability to account for external factors like weather or to share information with other vehicles. The **IoT-based approaches in recent research** (e.g., William & Olayemi, 2025 [1]) point towards more comprehensive systems: they envision networks of sensors and cameras monitoring speed and possibly adjusting limits based on conditions. However, many of these are still at a conceptual or pilot stage; they focus on enforcement or infrastructure communication rather than the direct in-vehicle driver experience and often assume advanced infrastructure that might not yet exist widely (like 5G coverage or smart traffic lights everywhere).

The **proposed system** in this research aims to bridge these gaps by integrating all the critical components into one framework. It will use **onboard sensors (camera, speed, weather)** to capture the immediate context of the vehicle, apply **intelligent algorithms** to assess risk (e.g., detecting if the driver is speeding given the current speed limit and weather conditions), and then employ **instant communication** channels to deliver a warning to the driver (and optionally to a central system or nearby vehicles). Importantly, this system is conceived with practical deployment in mind – leveraging affordable hardware (microcontrollers, simple sensors) and existing mobile networks (GSM) or ad-hoc networks, making it suitable for deployment in Sri Lanka's transport sector where cost and infrastructure are considerations.

By focusing on this gap – the lack of a **unified, real-time driver alert system that is context-aware** – our research targets a solution that could significantly reduce accidents. It is not merely about detecting speeding or issuing tickets; it is about preventing the dangerous situation from escalating by informing the driver at the critical moment. This requires contributions in multiple areas: reliable detection (computer vision for signs, sensors for weather), fast and accurate data processing (embedded systems and algorithms), and effective alerting (human factors in how to best warn the driver without

causing distraction). The following sections detail the research problem and the approach to tackling these challenges.

1.4. Research Problem

The core research problem can be summarized as follows: **How can we effectively utilize IoT and on-board sensing to monitor a vehicle's compliance with speed limits and adjust to real-time road conditions (like weather), and promptly warn the driver to mitigate risk?** In operational terms, this means developing a system that *accurately detects* the current speed limit and road condition, *determines* if the driver's current behavior is unsafe, and *delivers* an immediate alert or intervention to encourage safer behavior. Achieving this involves several interrelated challenges:

- **Robust Detection:** First and foremost, the system must reliably perceive its environment. This includes computer vision challenges in detecting and reading speed limit signs under various conditions. Road signs may be obscured, poorly lit at night, or blurred by heavy rain. Ensuring the camera-based sign recognition works with high accuracy in real time is a significant technical hurdle. Similarly, assessing weather conditions accurately is non-trivial – e.g., differentiating between light rain (where a small caution might suffice) and heavy rain or fog (which warrants urgent warnings). We must decide on appropriate sensors or data sources for weather (rain sensors, light sensors for visibility, or external data feeds) and fuse that information effectively. False detections – such as misreading a sign or mistakenly classifying a dry road as wet – could lead to incorrect alerts, which might annoy the driver or cause distrust in the system. Thus, a key problem is refining sensor and algorithm performance to **minimize false positives and negatives**, ensuring the system's decisions are both sensitive and specific to real dangers.
- **Real-Time Processing and Communication:** The value of the system lies in immediacy – a warning given even half a minute too late might fail to prevent an accident. Therefore, upon detecting a potential risk (e.g., the vehicle is over-speeding for the current zone, or a sudden downpour has begun while the vehicle is at high speed), the system needs to activate alerts almost instantaneously. This imposes constraints on processing time and communication latency. Implementing efficient algorithms on an embedded platform (like an Arduino or Raspberry Pi) is challenging due to limited computational resources, especially for image processing tasks. We might need to explore optimizations or edge-computing techniques to ensure the camera feed is analyzed in real time. If the design includes sending data to a cloud or central server (for logging or further analysis), we must ensure that this does not slow down the immediate feedback to the driver. The communication module (GSM or VANET radio) must transmit critical messages quickly and reliably, which can be difficult in areas with poor network coverage or high latency.

Ensuring **rapid signal processing and transmission** is thus a central technical problem – any bottleneck in this chain could render the "real-time" claim invalid.

- **Contextual Decision Logic:** Another challenge is determining *when* to alert and *when not to*. The system should ideally only alert the driver when a meaningful risk is present, to avoid "alarm fatigue." For example, if the driver briefly exceeds the speed limit by a tiny margin while overtaking, should the system trigger a warning or recognize it as a transient, low-risk event? Similarly, during light rain, the system might tolerate a smaller speed reduction, whereas heavy rain should trigger an emphatic alert if the driver doesn't slow down sufficiently. We need to develop logic (rules or machine learning models) that interpret the sensor data contextually – combining speed, sign limit, and weather into a **risk level assessment**. Defining these thresholds and behaviors is part of the research problem, potentially requiring analysis of accident data or traffic safety guidelines (e.g., how much should one slow down in heavy rain). The system's intelligence lies in its ability to emulate a cautious expert driver's judgment in various scenarios. Any automated warning system must also consider human factors: alerts should be clear, actionable, and not overly frequent. If alerts are too sensitive, drivers might start ignoring or disabling the system; if too insensitive, the system fails its purpose. Striking the right balance is a nuanced problem to solve.
- **Integration and Reliability:** Finally, integrating all components (camera, speed sensor, weather sensor, processing unit, communication module) into a **cohesive, reliable unit** presents practical challenges. The system will likely be subjected to vibrations, temperature changes inside a vehicle, and electrical noise from the vehicle's systems. Hardware reliability (e.g., camera lens getting dirty or rain sensor malfunctioning) can affect performance. We must consider failsafe measures – for instance, if one sensor fails, can the system still operate in a degraded mode? Also, power management is a consideration; the device should not drain the vehicle battery excessively nor be too power-hungry to run on a small onboard power source. Ensuring the device is robust enough for real-world usage (as opposed to just working in lab conditions) is a final critical challenge.

In essence, the research problem spans sensing, computation, and human interaction aspects. It is about optimizing technology to *augment the driver's situational awareness*. By refining sensor accuracy, speeding up processing, and tuning the system's responses, we aim to create a reliable co-pilot that continuously watches for risks and communicates them instantly. Overcoming these challenges will require a multidisciplinary approach, pulling from computer vision, embedded systems, wireless communications, and traffic safety expertise. The subsequent sections will outline the objectives derived from this problem and the methodology to address them.

2. OBJECTIVES

2.1 Main Objective

The **main objective** of this project is to **develop an IoT-based real-time driver assistance system that enhances road safety by monitoring vehicle speed and road conditions, and proactively alerting drivers of potential risks**. In simpler terms, the goal is to create a device and accompanying network that will help prevent accidents in Sri Lanka by ensuring drivers are aware of and adhere to safe speeds given the current road situation (posted speed limits, weather, etc.). This objective can be unpacked in context as follows:

The envisioned IoT device is an on-board unit installed in a vehicle (for example, a bus or truck) that continuously watches the road ahead and the vehicle's performance. It integrates a suite of sensors and modules to achieve its safety function. A forward-facing **camera** serves as the "eyes" of the system, using computer vision to recognize road signs – particularly speed limit signs. For instance, as a bus travels, the camera might spot a sign indicating the speed limit is 50 km/h; the system will interpret this and compare it to the bus's actual speed obtained from a speed sensor or GPS module. If the bus is found to be traveling above 50 km/h in this zone, the device will immediately trigger a warning to the driver, such as a beeping alarm or a voice message, advising them to slow down. This real-time feedback loop encourages compliance with traffic regulations and can prevent accidents caused by inadvertent speeding or driver oversight of signage.

In addition to sign detection, the device monitors **environmental conditions** in real time. It can either use on-board sensors (like a rain sensor to detect precipitation and an ambient light sensor to gauge visibility) or pull data from a weather service based on the vehicle's GPS location. By doing so, the system knows if it's raining heavily, if the road might be wet, or if there is fog or low light. Why is this important? Because even when a driver is within the posted speed limit, certain conditions warrant extra caution. For example, driving at 50 km/h on a highway during a torrential downpour can be just as risky as speeding on a clear day. The device's intelligence lies in adjusting its warnings based on such context. If heavy rain is detected, the system might effectively lower the "safe" speed threshold and alert the driver to slow down even if they are technically at or slightly below the posted limit. Conversely, in clear conditions it would be less intrusive. Through this adaptive approach, the device serves as a constant safety companion that guides the driver to appropriate behavior for current conditions.

A critical aspect of the system is its **communication capability**. The term "IoT" implies that our device is not working in isolation – it can connect and share data. Using a built-in GSM module (or alternatively a Wi-Fi/LoRa module depending on infrastructure), the device can send and receive information to a central server or even directly to other vehicles (forming a Vehicular Ad-hoc Network, VANET). This opens up several safety-enhancing possibilities. For example, if one vehicle's system detects a sudden hazardous event – say it encounters a flooded section of road or an accident blocking the highway – it can instantly transmit a warning message to approaching vehicles equipped with similar devices, or to a central traffic management center

which can then disseminate alerts via digital road signs or emergency broadcasts. In the context of Sri Lanka, this could mean that if a bus ahead detects a danger, buses coming behind (or even connected infrastructure) are made aware in advance, buying drivers precious reaction time. Additionally, the device can log data such as locations of frequent overspeed occurrences or near-miss alerts to a cloud database for analysis. Stakeholders like fleet operators or authorities could use a companion **Android mobile application** to monitor this data in real time or get notified when one of their vehicles repeatedly triggers warnings (indicative of risky driving that may need intervention).

Concretely, implementing this main objective will result in a prototype system demonstrating scenarios such as: A bus traveling on a highway enters a zone with a lower speed limit – the driver misses the sign, but our camera doesn't and promptly beeps until the driver slows to compliance. Later, the bus approaches an area where a sudden storm has made roads slippery; the system, noting heavy rainfall via its sensor, checks that the driver has reduced speed sufficiently. If not, it issues a caution like "**Heavy rain – slow down**" on a display or speaker. Simultaneously, it might send a message to the central system that "Bus #5 at location X is experiencing heavy rain", which could alert other nearby vehicles or at least record the event. As the weather clears or the bus leaves the risky zone, the system will cease the warnings, avoiding unnecessary distraction once the risk subsides.

In essence, the main objective encapsulates a **proactive safety system for road travel**. By blending local sensing (on the vehicle) with networked intelligence (IoT communication), the project seeks to significantly reduce the likelihood of accidents caused by human error or unanticipated conditions. This directly contributes to saving lives and reducing injuries on Sri Lankan roads, aligning with broader road safety goals. It's about moving from reactive measures (like post-accident responses or fines after a law is broken) to a preventive paradigm where the technology assists the human driver in making safe decisions in the moment.

2.2 Specific Objectives

To achieve the overarching goal of the project, several **specific objectives** have been identified. These break down the main objective into targeted, actionable components in the proposed system:

[1] Implement a Vehicular Ad-Hoc Network (VANET) capability for data sharing: Develop the system's communication subsystem such that vehicles equipped with the device can communicate with each other and with a central monitoring unit. This involves configuring modules (e.g., Wi-Fi or dedicated short-range radios) and communication protocols to form an ad-hoc network among vehicles. The VANET will facilitate the exchange of safety information—like broadcasting alerts about hazardous road conditions or sudden braking events—to nearby vehicles. By enabling V2V (vehicle-to-vehicle) and V2I (vehicle-to-infrastructure) communication, this objective ensures our system isn't isolated to a single vehicle's perspective but contributes to a networked safety net. For example, if one bus detects an obstacle on the road, it can send a warning via VANET to following buses so they can react sooner.

[2] Real-time transmission of alerts to stakeholders upon risk detection: Configure the system to promptly transmit relevant information to stakeholders as soon as a risk is detected. "Stakeholders" here primarily include the vehicle's driver, but can also extend to others like fleet managers or traffic authorities through the connected app or network. When the device receives a trigger (e.g., the camera identifies a speed limit and finds the vehicle is over-speeding, or a rain sensor indicates dangerous conditions), it should within seconds send out an alert. For the driver, this means an in-cabin visual or auditory alert urging corrective action. Simultaneously, a succinct data message (containing location, nature of alert, etc.) can be sent to a cloud server or traffic control center. Monitoring personnel could use this information to take further action if needed (for example, dispatching highway patrol to a danger spot, or simply keeping logs of incidents). The goal is to **minimize the latency** between detection and notification so that responses to potential risks are as rapid as possible.

[3] Smart deactivation of warnings and integration of a system database: Create logic for the system to deactivate or cease the warning signals once the hazard or trigger condition is resolved, and maintain a database of events for analysis. It's important that if a driver responds correctly (e.g., slows down to the speed limit, or the heavy rain subsides), the system recognizes this and turns off the alert to avoid annoying or distracting the driver unnecessarily. This objective includes implementing a hysteresis or condition-check mechanism so that alerts are only active during the window of actual risk. Moreover, this objective covers building a simple database (likely SQL-based) that the system updates with each alert event: recording data such as timestamp, location (via GPS), type of alert (over-speed, weather caution), and whether/how quickly it was resolved. This database can be on the device (and later synced) or directly on a cloud server via GSM. The stored data will be crucial for evaluating system performance, identifying recurring trouble spots (e.g., locations where drivers often ignore a certain speed limit until alerted), and could serve as evidence in case of incidents.

[4] Accurate detection of road signs and activation of appropriate signal alerts: Develop and train the core detection algorithms to reliably identify target events (like an elephant presence in the earlier scenario, but for our case: road signs, speed deviations, and unsafe conditions) and trigger the necessary warning signals. This involves the computer vision component – likely using techniques such as image processing or machine learning to recognize speed limit signs from the camera feed. The system must differentiate speed limit signs from other signs and correctly interpret the numeric value (e.g., "50" for 50 km/h). Additionally, this objective encompasses the detection of environmental conditions via sensor data (interpreting sensor readings to conclude "rain" or "fog" conditions). Once a detection is made (say a "50 km/h" sign is recognized), the system's control logic compares it with the vehicle's actual speed from the speed sensor. If a rule is violated (vehicle is above 50 km/h by a certain tolerance), the system **activates signal alerts** – which could be a flashing LED on the dashboard, a buzzer sound, or a spoken alert through a speaker. Similarly, if a "dangerous condition" like heavy rain is detected and the vehicle exceeds the recommended safe speed for that condition, the system should activate a warning. Essentially, this objective is about tying together the sensing and actuation: making sure that the detection of a risky situation leads to an immediate and appropriate alert output. The alerts should be noticeable and clear to the driver (for example, a red warning light accompanied by a sound for overspeed, or a different icon/sound for weather caution) so that they prompt the desired action (slowing down, etc.). Fine-tuning the sensitivity (when to trigger) and modality (how to alert) will be part of this objective to ensure effectiveness without causing distraction or panic.

Each of these specific objectives contributes to the main goal by focusing on a particular aspect of the system. Objective [1] expands the system's reach through networking, [2] and [4] ensure the core functionality of detection and alerting works in real time, and [3] adds intelligence in managing alerts and collecting data for continuous improvement. Together, fulfilling these objectives will result in a comprehensive solution that monitors, communicates, and advises in real time to enhance driver safety on the roads.

3. METHODOLOGY

3.1 Project Overview

At a high level, the methodology involves designing an integrated hardware and software system that can be mounted on a vehicle and perform the required monitoring and alerting functions. The project is structured in iterative phases, each addressing key components of the system. Figure 2 provides an overview diagram of the system architecture and data flow, illustrating how information moves from sensor detection to driver notification and network communication.

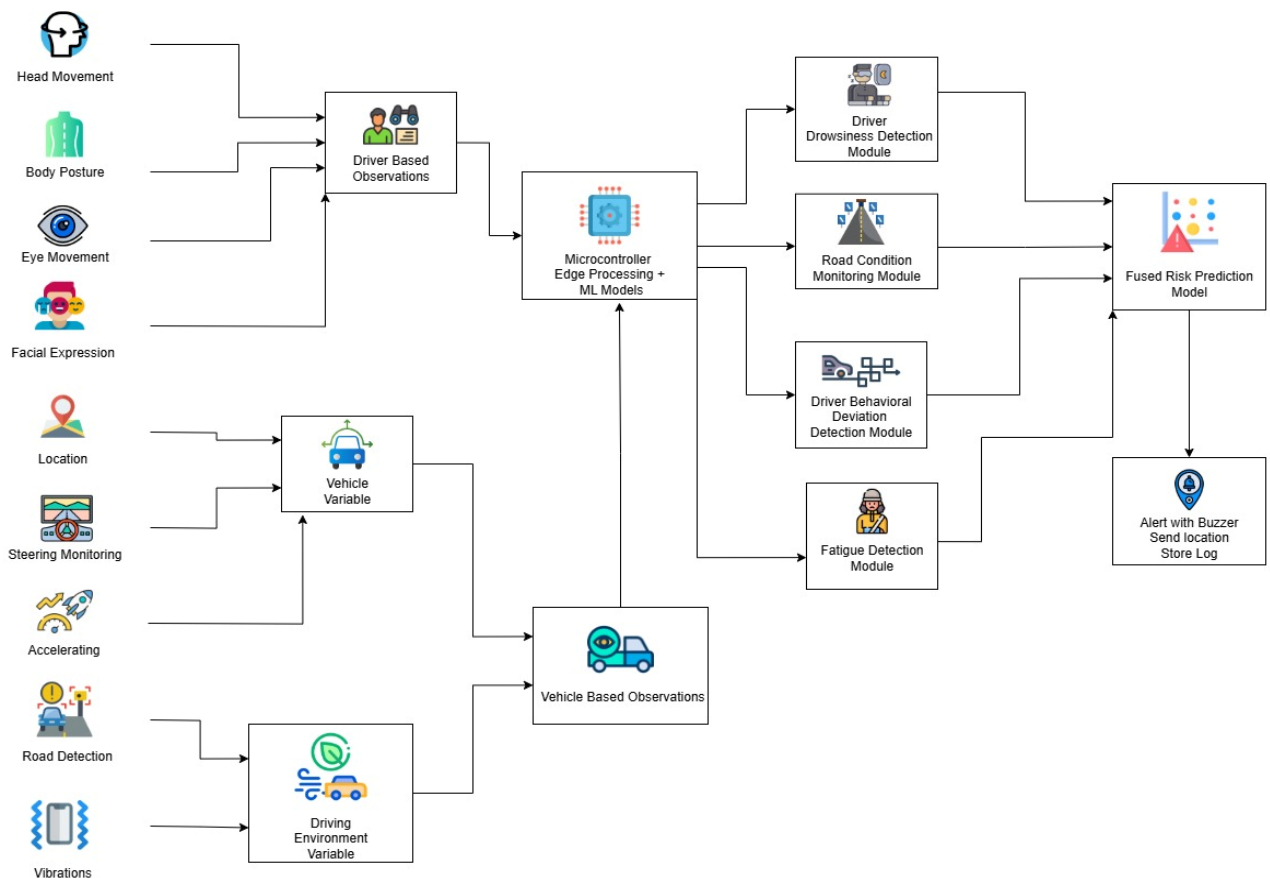


Figure 2: Project Overview Diagram

The **proposed system** consists of several main components working in unison: a sensing module (camera and other sensors), a processing and control unit (microcontroller or embedded

computer), a communication module (for GSM/VANET connectivity), and an alert interface for the driver. The methodology follows a pipeline approach:

1. **Sensor Data Acquisition:** The first step is capturing raw data from the environment. A camera will continuously capture the road ahead, while additional sensors (like a GPS receiver, accelerometer, rain detector, etc.) will gather other contextual data. The placement and calibration of these sensors are crucial – for example, the camera needs an unobstructed view of the road and might be mounted near the windshield similar to where modern dashcams or ADAS cameras are placed. We plan to use a dual-sensor approach for certain detections; for instance, combining a rain sensor's readings with windshield wiper status or traction control signals (if accessible) to robustly infer the road wetness condition.

2. **Data Processing & Detection:** The raw data from sensors are fed into the processing unit (an Arduino board or Raspberry Pi). Here, our algorithms analyze the data in real-time. For the camera input, we employ image processing techniques to detect traffic signs. This could involve trained machine learning models (like a CNN that can recognize speed limit sign patterns) or classical methods (such as color and shape filtering to find circular signs with numbers). The output of this analysis is a digital interpretation of what the camera sees – e.g., "Speed limit 40 km/h sign detected" or "No relevant sign detected right now." In parallel, the system checks the vehicle's current speed (from GPS or a speed sensor) and reads the weather sensor values. The processing algorithm combines these inputs to determine if there is a **trigger condition**. A trigger condition could be defined by rules such as:

    - *Overspeeding:* Vehicle speed > detected speed limit + tolerance (say 5 km/h).
    - *Weather hazard:* Vehicle speed > safe speed under current weather (which might be a percentage of the normal speed limit or an absolute threshold if visibility is low).
    - *Sudden event:* Abrupt braking or obstacle detection (if we extend the system with, say, an ultrasonic sensor to detect distance to vehicle in front, though that's beyond core scope currently).

If a trigger condition is met, the logic proceeds to the next steps; if not, the system continues monitoring passively.

3. **Alert Generation & Response:** When the system identifies a risky situation (like overspeeding), it immediately generates an alert. For the driver, an **in-vehicle alert** is activated. The prototype will likely use a buzzer or speaker for audio alerts and possibly an LED indicator or small display for visual feedback. The alert is designed to be attention-grabbing but not startling to the point of causing panic. For example, a repeated beeping tone plus a dashboard LED might indicate "overspeed – slow down," whereas a different tone/pattern might be used for a weather warning. The methodology includes testing different alert methods (sound volume, frequency, voice vs. tone, etc.) to find an

effective mode. The system will also log this event internally (timestamp, type of alert) as part of building the device's memory of incidents.

4. **Communication & Data Sharing:** Simultaneously with alerting the driver, the system's communication module kicks in to share data. Using the GSM module, a pre-formatted SMS or data packet could be sent to a server or predefined contacts (for example, a fleet management office or a cloud database). For instance, if the bus continues to speed excessively despite warnings, the system might send a message to the central system with the bus ID and location, flagging a violation or danger. This allows remote stakeholders to intervene if needed (maybe contacting the driver via radio or logging the infraction for later review). In a more immediate sense, if we have multiple vehicles equipped, the VANET capability (Objective [1]) would broadcast a short-range alert to other vehicles in the vicinity. The methodology for VANET involves configuring all devices to listen on a common wireless channel for alert messages and to relay or display them accordingly. Essentially, each vehicle becomes a node in a mesh network that can propagate important warnings (like an accident ahead or an animal crossing on a rural road as additional context).

5. **Feedback Loop & Deactivation:** After issuing an alert, the system doesn't stop there; it continuously monitors whether the situation improves. If the driver slows down to an appropriate speed, the system should recognize the compliance and then **deactivate the alert** (the buzzer stops, the warning light turns off). This requires a small state machine within the control logic – once triggered, it will keep alerting until the conditions return to safe ranges for a certain minimum time. This prevents the alert from flickering on and off if the driver hovers around the threshold. The methodology includes calibrating this by testing, for example, how quickly drivers typically respond and ensuring the alert stops promptly once they have done the needful, to positively reinforce the behavior and not irritate them. All the while, if conditions worsen or a new trigger arises, the system adapts. It is essentially a **continuous loop** of sense -> analyze -> act -> communicate, running as long as the vehicle is in operation.

Throughout this process, an emphasis is placed on **formalizing the operations** to ensure reliability. This means that for each functional block (say, the sign recognition algorithm), we will likely follow a mini development cycle of design, simulate/test on sample data, then implement on hardware and fine-tune. The project will also involve setting up **testing scenarios**: for example, simulating a stretch of road with various speed limit signs (perhaps by showing the camera recordings or actually driving along a test route) to validate that the camera correctly picks up each sign and triggers the right alert. Similarly, testing under different weather (which might be simulated by spraying water in front of the sensor or running the system during actual rain) will be done to ensure the sensor and logic respond appropriately.

In conclusion, the project overview encapsulates building a **prototype system** and gradually refining its components through testing and iteration. By breaking the problem into sensing, processing, alerting, and networking, the methodology ensures each aspect can be developed and verified step by step before integrating everything into the final solution.

3.2 System Overview Diagram

The system's architecture can be visualized as a set of interconnected modules that collectively enable real-time monitoring and risk prediction. In the *System Overview Diagram* (Figure 2), we depict the flow of information and the key components involved. To describe it verbally: the **dual sensing mechanism** is at the forefront. One branch of sensing is the **camera module**, which provides continuous video feed of the road ahead. The other branch is a collection of **auxiliary sensors and data inputs** – this includes a speed sensing mechanism (which could be a direct speedometer interface or GPS-based calculation), a weather sensing mechanism (rain detector, possibly temperature/humidity or fog sensor, and ambient light sensor for darkness), and position tracking (GPS location for context like mapping to known speed zones or weather API).

These inputs all feed into the central **Processing Unit** (likely represented as an Arduino or microcontroller in the diagram). The processing unit is programmed with algorithms to interpret each input:

- The camera feeds through a **Sign Recognition Algorithm**. This is shown in the diagram perhaps as a sub-block labeled "Image Processing Module". It outputs the current detected speed limit or other warning signs.
- The speed sensor input goes into a **Speed Monitoring Module**, which simply provides the current vehicle speed in real time.
- The weather sensors feed a **Condition Monitoring Module**, which might output a qualitative assessment like "NORMAL" vs "HEAVY RAIN" or a set of parameters (rain intensity level, light level, etc.).
- Additionally, the system might have a **Location Module** that uses GPS coordinates to fetch external data if needed (like querying a cloud service for live weather info or known road hazard data). In the diagram, this might be indicated by a connection to an external data source (via the GSM network) for something like weather updates or traffic alerts from authorities.

Once the processing unit has these interpreted inputs, it runs the **Decision Logic**. This logic can be thought of as the brain that decides whether an alert needs to be raised. For example, the logic checks:

1. Is a valid speed limit detected? If not, perhaps use a default or previously known limit for the road.
2. If yes, compare vehicle speed to that limit. Calculate difference.

3. Check weather status: if heavy rain/fog, maybe effectively lower the acceptable speed threshold by a certain factor.
4. Determine if the vehicle is significantly over the allowed speed (taking weather into account). Also, the logic might consider acceleration or break patterns (if an accelerometer is included) to catch if, say, the driver is approaching too fast towards a slower zone.
5. If criteria are met, mark this as a "Risk Alert Condition."

In the diagram, from the Decision Logic, one path goes to the **Alert System** (within the vehicle) and another path goes to the **Communication System**. The Alert System could be represented by icons of a speaker and a flashing light on a dashboard, indicating that it will produce an audible alarm and a visual cue to the driver. In real implementation, this could be an LED array or an LCD display that shows a message like "Slow Down!" alongside a buzzer. The **1.5-second delay** concept (mentioned analogously in the Elephant monitoring section of the original doc) might also apply here: for instance, the system might wait for confirmation over a short interval (like requiring that both the camera detection and speed reading consistently indicate a violation for, say, 1.5 seconds) before triggering the alert. This prevents very brief false triggers (like momentarily reading a sign from a side road or a one-second speed spike) from causing a flurry of unnecessary warnings. In our diagram, this could be depicted as a condition that both "Sensor A = HIGH" and "Sensor B = HIGH" for a certain time leads to activation, similar to how the elephant system used dual IR beams to confirm presence.

Simultaneously, the Communication System kicks in. In the diagram, this would be a GSM module icon (maybe a tower symbol for cellular network) or a network cloud symbol for VANET. The GSM module executing AT commands indicates it can send SMS or data; so in the context of our project, once an alert is triggered, the GSM module might send out an SMS alert. For example, a text could be sent to a cloud database or a monitoring phone number with content like: "Bus ID 05: Overspeed alert at location X (Speed 60 in 40 zone)". If we have a mobile application set up, the app would be listening for such updates (perhaps via an internet API or direct SMS reception) and then display it to relevant stakeholders. The system overview shows this connection: the **Mobile Application** is synced with the database or directly receives the info, providing real-time updates on the smartphone. This would be particularly useful for fleet managers or for evaluating driver performance over time.

Now, the integration of a **Vehicle Ad-Hoc Network (VANET)** adds another layer to the system diagram. A VANET would allow direct car-to-car communication without necessarily going through the cellular network (especially important if we consider scenarios with poor GSM signal or if we want to alert nearby vehicles faster than a cloud round-trip). In the diagram, vehicles could be shown exchanging signals. For instance, if our device recognizes a hazard, it could emit a short-range broadcast (via Wi-Fi or radio) that other vehicles' devices pick up. The diagram might illustrate two vehicles and an arrow between them representing this exchange. The VANET module ensures **dynamic adjustments**: for example, if the traffic ahead is slowing rapidly (one vehicle's accelerometer could detect a sudden deceleration), a following vehicle's

system might get that data via VANET and prepare to warn its driver even before the driver sees the brake lights ahead.

In summary, the system overview diagram underscores how multiple components – **sensors, processor, communication, and user interface** – are orchestrated. Data flows from **physical world (road & environment)** -> **sensors** -> **processor (decision making)** -> **actuators (alerts, communication)** -> back to **users (driver, stakeholders)**. By designing the system in modular blocks, the methodology allows focused development and testing of each part:

- We'll test the sign detection module independently with sample images,
- Test the speed reading and threshold logic with simulated data,
- Test the GSM communication by sending test messages,
- Finally test end-to-end on a closed course or controlled setting.

The robust design shown in the diagram is intended to maximize safety: there's redundancy (multiple sensor inputs), verification delays to reduce falses, and multi-channel alerting (local to driver and remote to others). This multi-layered approach is aimed at creating a **seamless channel for quick and well-informed responses**, as mentioned in the abstract. Essentially, the vehicle becomes a smart node that knows its environment and can both assist its own driver and collaborate with the broader traffic network to prevent accidents.

3.3 Individual Component

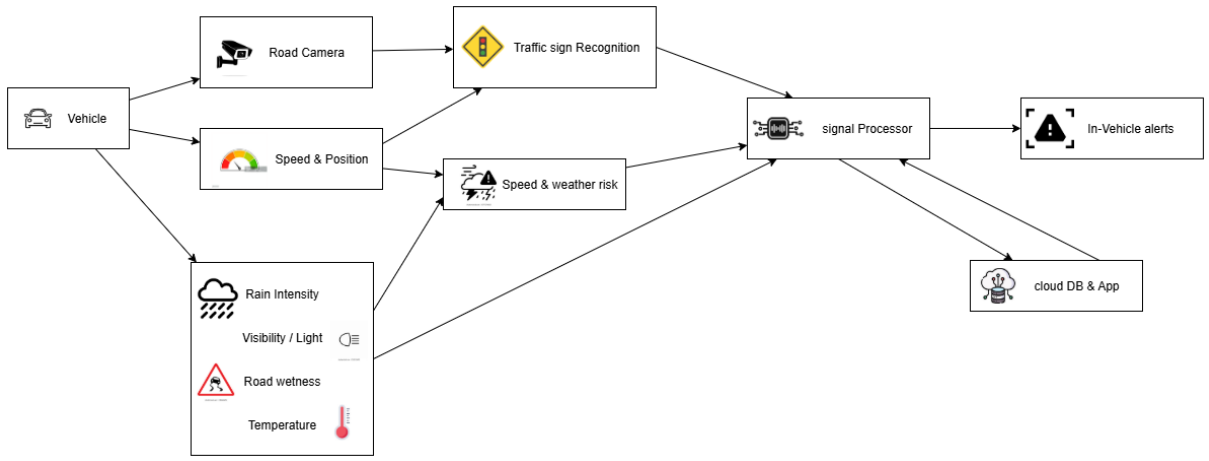


Figure 3: Project Overview Diagram

3.3.2 The flow of the system diagram – Individual Component

1. **Initial Detection by Camera (Vision Component):** The process begins with the vision system scanning the road. The camera continuously captures frames of the road ahead. Suppose the vehicle approaches a road sign – the image processing algorithm kicks in to analyze each frame for familiar patterns that match traffic signs (especially the circular shape and specific numerals of speed limit signs). When a speed limit sign is recognized in the frame, the system immediately notes the speed value (for example, detecting a "40" on the sign indicating 40 km/h limit). This detection is the trigger for the next steps. It is important to note that lighting and distance factors are considered; the algorithm might use morphological operations and feature matching so that it can detect signs even from some distance, giving the driver early notice.

2. **Signal Relay to Processing Unit (Arduino Board):** Upon detecting a sign or a condition, the camera module (which could be a smart camera or the feed processed by the same board if using something like an Arduino with OpenCV on a companion microprocessor) sends a digital signal or data to the central Arduino (or microcontroller). For instance, it might update a variable in memory like current_speed_limit = 40. Similarly, other sensors relay their instantaneous readings: the GPS module might update current_vehicle_speed = 55 km/h (obtained either via the vehicle's speedometer interface or calculated from GPS movement), and a rain sensor might set a flag rain_detected = TRUE if it senses moisture.

3. **Processing and Decision (Arduino logic):** The Arduino now has all the pieces of information needed to make a decision. Using a predefined algorithm (essentially an *if-*

else decision tree or a slightly more advanced fuzzy logic system for combining factors), it evaluates:

- Compare current_vehicle_speed against current_speed_limit. In our hypothetical scenario, the bus is going 55 in a 40 zone, which is clearly above the threshold (if we allow maybe a 5 km/h grace, 55 is still well above 45). So an overspeed condition is detected.
- Check the rain_detected flag (and potentially other weather inputs). If rain is true, the urgency is even higher. The system might lower the threshold for acceptable speed. In this case, even without adjusting for rain, the overspeed is significant. With rain present, definitely it's deemed hazardous.
- The Arduino logic determines that an alert condition exists (vehicle significantly exceeding safe speed under current conditions). It sets an internal state like alert_overspeed = TRUE.

4. **Activation of GSM Module for Alert Transmission:** Once the decision is made to alert, the Arduino communicates with the GSM module (which is connected via serial interface typically). It uses AT commands (attention commands) to either send an SMS or a GPRS data packet. For example, the Arduino might execute:
5. AT+CMGS="+94XXXXXXXX"

   > "Alert: Bus 07 overspeed 55km/h in 40 zone, Rain: Yes, Location: [lat,long]"

This would send an SMS to a preset number (which could be a service or person). Alternatively, using GPRS, it might hit a web API to log this event in a database. The methodology ensures that this communication is done in a non-blocking way if possible (so the Arduino doesn't freeze waiting for network response).

In parallel or alternatively, if a VANET is configured, the Arduino formats a short broadcast message (for instance, a UDP packet over Wi-Fi containing something like ALERT, location, type) and sends it out via a Wi-Fi or XBee module. Nearby vehicles' systems, if within range, receive this broadcast and their logic will know "there is a hazard/overspeed up ahead at X location or by vehicle Y." They might then display a message like "Vehicle ahead speeding in rain – potential hazard" or simply use it as context to become more vigilant (this part can be scaled in complexity as needed).

6. **Mobile App Update (Stakeholder Notification):** The transmitted information is received on the other end. If SMS, perhaps the traffic control center's phone or an automated server reads it. If GPRS to a cloud, the cloud logs it. The Android mobile application that stakeholders have will periodically query this cloud database or listen for push notifications. Upon receiving the new event, it can show a notification such as "Bus 07: Overspeed warning at 3:45 PM near Kandy Road". This keeps remote stakeholders in the loop in real time. Our methodology for the app involves using a simple interface with

a list or map of alerts, enabling a supervisor to potentially call the driver or take note for later coaching.

7. **Driver Alert Mechanism (In-Vehicle Alerts):** Simultaneously (and most immediately critical), the Arduino triggers the on-board alert for the driver. If alert_overspeed = TRUE, it might set a digital output high that is connected to a buzzer circuit and an LED. The buzzer starts beeping and the LED labeled "Slow Down" lights up on the dashboard. In our design, we considered human factors: the alert likely needs to be annoying enough to catch attention but not so alarming that it startles the driver into a worse mistake. A repetitive chime or moderate buzzer is suitable. The driver, hearing and seeing these warnings, is now aware that they are exceeding a safe speed. In response, a responsible driver would ease off the accelerator to reduce speed. This feedback loop is exactly what we want – a quick correction to avert danger.
8. **Monitoring for Resolution and Deactivation:** The system doesn't stop at issuing an alert; it continues to monitor the situation. Now suppose the driver slows down from 55 to 40 within a few seconds of the alert. The speed sensor picks this up, updating current_vehicle_speed in real time. The Arduino logic sees that the condition vehicle_speed > speed_limit is no longer true (or at least once it drops under threshold). After a brief confirmation period (maybe 1-2 seconds to ensure the speed is stably under the limit and the driver hasn't just tapped the brakes and then sped up again), the Arduino will reset the alert state. It turns off the buzzer and LED. The driver notices the warning has ceased, signifying they are now in compliance. If the road conditions are still poor due to rain, the system might continue to advise a general caution (perhaps a different indicator or a toned-down alert until the rain stops). The key is that **the alerts are context-sensitive and temporary** – only present during the unsafe condition.
9. **Logging the Event:** In the background, the system logs this incident. It might note that an overspeed alert was triggered at X time for Y duration until driver complied. This data could be stored in non-volatile memory or immediately sent to the cloud as part of the earlier communication. Such logs are later useful to analyze driving patterns (e.g., if a particular route or driver triggers frequent alerts).

This detailed flow demonstrates how the individual components (camera, sensors, Arduino, GSM, alerts, app) orchestrate together. By meticulously following this structured methodology, the integrated system ensures efficient detection and dissemination of crucial information about the vehicle's status and environment. Each component has a well-defined role and interface:

- The camera "watches" for signs.
- The Arduino "thinks" and decides based on multi-sensor input.
- The GSM/VANET modules "talk" to the outside world.
- The buzzer/LED "tell" the driver what to do.

This clear separation and flow also makes the system modular and easier to troubleshoot or improve. For instance, if sign recognition isn't accurate enough, we can focus on improving the camera algorithm without altering how alerts are transmitted or displayed (as long as the output format remains the same). Similarly, if we find GSM communication slow, we could upgrade to a faster network module or adjust the frequency of sending without affecting detection.

Overall, by adhering to this methodology, the system aims to **orchestrate swift detection and communication** of any unsafe driving scenario. It is proactive – catching issues as they happen – and reactive in a controlled, helpful manner to guide the driver towards safety. This structured flow is at the heart of how the project will be implemented and tested.

3.4 Tools and Technologies

Table 2: Tools and Technologies Utilized in the System

Tools	Technologies	Explanation
Arduino Boards		The Arduino board serves as the central microcontroller platform in our system. It is a compact yet powerful board that comes with a microcontroller unit (MCU) and a host of input/output pins for interfacing with sensors and actuators. Arduino is renowned for its simplicity and flexibility, which makes it ideal for rapid prototyping. In this project, the Arduino is programmed to read sensor inputs (camera module output, speed sensor, rain sensor, etc.), execute the decision-making algorithm, and then activate outputs (buzzers, LEDs, communication modules) accordingly. The choice of Arduino is motivated by its user-friendly development environment and a large community support base, which provides plenty of libraries and example code (for things like serial communication, sensor integration, etc.). This allows even complex tasks like parsing GPS data or sending SMS via a GSM module to be implemented relatively easily. The Arduino's real-time processing capabilities ensure that sensor data is handled promptly, which is crucial for our application. Multiple variants of Arduino (such as Arduino Uno, Mega, or Nano) can be chosen

		based on memory and pin requirements; for instance, if we need more memory for image processing or more pins for additional sensors, an Arduino Mega might be used. Overall, the Arduino forms the "brain" of the system, coordinating all other components.
	Camera Module (with Computer Vision)	The camera (or image sensor) is a pivotal component responsible for capturing visual data of the road ahead. In our setup, we likely use a camera module (such as the OV7670 paired with Arduino, or a Pi Camera if using a Raspberry Pi for the vision part) to continuously take images or video frames. The role of the camera is to detect road signs and possibly other visual cues like lane markings or obstacles. Because the raw camera data is just images, we employ computer vision techniques (either directly on the microcontroller if capable, or on an attached processor) to interpret these images. Technologies like OpenCV (Open Source Computer Vision library) can be utilized if we have a platform that supports it (Arduino itself might be too limited for heavy image processing, so sometimes an embedded Linux board like Raspberry Pi is used in tandem). Through these techniques, the system can recognize specific patterns – particularly the shape and numbers of speed limit signs. Recent advances allow even small embedded systems to run trained deep learning models (e.g., using frameworks like TensorFlow Lite or Edge Impulse) for object detection and recognition with reasonable speed. In summary, the camera module combined with computer vision algorithms gives the system "eyes" and the ability to understand the visual road environment, which is crucial for identifying speed limits and warning signs in real time.
	GPS Module (Speed and Location Sensor)	A GPS (Global Positioning System) module is utilized to obtain real-time data on the vehicle's location and movement. By receiving signals from satellites, the GPS module can determine the vehicle's geographic coordinates and often its speed and heading as well. In this project, the GPS module serves a dual purpose. Firstly, it provides the **vehicle's speed** by calculating the rate of

		change of its position (or directly reading the speed if the module outputs it). This offers an independent way to gauge how fast the vehicle is moving, supplementing or replacing a direct speedometer reading. Secondly, the GPS gives location data, which can be used for multiple enhancements: for example, the system can cross-reference the coordinates with a map of speed limits (if such data is available) as a redundancy to camera detection; or send location info in alerts so that stakeholders know where an incident is occurring. Additionally, location data allows integration with external services – for instance, querying a weather API for the precise area the vehicle is in, to get detailed weather conditions (beyond what on-board sensors provide). The GPS module communicates with the Arduino via serial (using protocols like NMEA sentences). Using a GPS in the system ensures that we have accurate speed data and context, which is vital for the core functionality and for situational awareness in communications.
	Rain/Weather Sensor	The rain sensor (or a broader weather sensor suite) is used to detect environmental conditions such as rain, moisture, or even fog. A typical rain sensor module might consist of a small board with exposed traces that change resistance or voltage when water droplets fall on it. The Arduino can read this via an analog or digital input to determine if precipitation is occurring and roughly how intense it is (e.g., by the amount of area covered by water drops on the sensor grid). In addition to a rain sensor, we might include an ambient light sensor (like an LDR – Light Dependent Resistor) to gauge daylight vs. darkness or fog (as heavy fog can reduce light levels) and a temperature/humidity sensor to gather more context (humidity can hint at fog potential, temperature near dew point etc.). These sensors together help the system infer conditions such as “wet road”, “reduced visibility”, or “normal clear weather”. This is critical for adjusting the system’s risk calculations – for instance, if rain is detected, the system will treat even a slight speeding more seriously because braking distances are longer on wet roads. The use

		of these sensors makes our system **environmentally aware**. They are relatively inexpensive and easy to interface (often analog signals or simple digital outputs for threshold-based rain sensors), which makes them a practical addition. Ensuring they are calibrated and maybe placed where they accurately catch rain (like on top of the vehicle or near the windshield) is part of the methodology. With a weather sensing capability, our device moves beyond just rule enforcement to adaptive risk management, because it understands the context in which the vehicle is operating.
	GSM Module (SIM800/SIM900 for Cellular Communication)	The GSM module is responsible for providing the system with connectivity to the cellular network. Modules such as SIM800 or SIM900 are commonly used with microcontrollers to send SMS messages, make voice calls, or even use GPRS data. In this project, the GSM module is chiefly used to send out text-based alerts and possibly to receive any remote commands (though primarily it will be transmitting data). When the Arduino triggers an alert, it uses AT commands to dial out a message via the GSM module – for example, sending an SMS to a central server or a designated phone number that carries information about the alert (vehicle ID, location, speed, etc.). The module requires a SIM card with an active cellular plan. By leveraging GSM, our system ensures that alerts can be communicated **over long distances**, irrespective of line-of-sight (unlike VANET which is short-range). This is crucial for notifying stakeholders who are not in the immediate vicinity of the vehicle – such as control centers or even emergency responders if needed. GSM coverage is quite extensive along populated areas and highways in Sri Lanka, which means this system will work in most places where vehicles travel. Additionally, using GPRS or even newer LTE modules, we could send data to internet servers in real time, enabling cloud logging and more complex analysis. The GSM module essentially turns our device into an IoT node that's part of the broader mobile network,

		allowing real-time data sharing and notifications across virtually any distance.
	SQL Database (Cloud or Local)	Structured Query Language (SQL) is used to manage the project's database, which can reside on the cloud or on a local server. The database will store records of events and system status updates. Every time the system triggers an alert or a significant event (like entering a new speed zone, or starting a trip, etc.), a new entry can be inserted into the database. This might include fields like timestamp, vehicle ID, event type (overspeed alert, harsh brake alert, etc.), speed, location, and status (resolved or ongoing). SQL databases are excellent for this because they allow structured storage and easy querying of data. For example, we can run queries to find "all overspeed alerts in the past week" or "the number of times a particular driver triggered warnings". In our architecture, the Arduino (via GSM) might directly use an HTTP API to send data to a cloud service where the SQL database resides (like a MySQL or PostgreSQL instance). Alternatively, the mobile application or a server receiving SMS can populate the database. We chose to incorporate a database because it provides a **persistent memory** of the system's operations which is invaluable for analysis and improvement. It enables generating reports, identifying problem routes or times, and even feeding into machine learning models in the future for predictive analytics. For instance, by analyzing the database, one could discover that a certain stretch of road frequently causes sudden braking alerts, indicating maybe a hidden hazard there. Thus, SQL-backed data management is a tool for both the development (tuning the system) and the stakeholder value (providing transparency and accountability of vehicle safety metrics).
	Android Application	An Android application is developed as part of the system to give stakeholders (like fleet managers, safety officers, or even the drivers themselves off-duty) an interface to interact with the system's data in real time. Android was chosen due to its wide use and flexibility in Sri Lanka – many users

		have Android smartphones, and it's relatively straightforward to develop for. The app will connect to the cloud database or directly receive communications from the vehicles (depending on system design). Key features of the app include: receiving push notifications of alerts (e.g., a popup when an overspeed event occurs), displaying a dashboard of current vehicle statuses (like showing a list of vehicles and whether they are within speed limits, and weather conditions around them), and possibly a map view to visualize where alerts are happening. The app can also store historical data or fetch it from the server to show trends and logs (like a timeline of today's alerts or a summary for each vehicle). Android's rich UI capabilities allow color-coded alerts, sounds, and interactive elements like acknowledging an alert or calling the driver directly from the app if needed. For security and privacy, the app would require authentication so that only authorized personnel can view or manage the data. Overall, the Android application serves as the **user-facing control panel** for the system, turning raw data into actionable insights and alerts for humans. It enhances the usability of the system by ensuring the information collected by our IoT device is accessible and understandable to those responsible for road safety and fleet operations.
	Arduino IDE	The Arduino Integrated Development Environment (IDE) is the primary software tool used to develop and upload code to the Arduino board. This IDE provides a user-friendly text editor for writing the C/C++ based Arduino code, a simple one-click compile/upload process, and a built-in serial monitor for debugging. We use the Arduino IDE to implement the logic for sensor reading, decision-making, and communications described in our methodology. Thanks to its extensive library support, we can include libraries (often available through the Arduino Library Manager) for handling devices like the GPS module (e.g., TinyGPS++ library), GSM module (SoftwareSerial or specific GSM libraries), and sensors (many come with their own libraries, simplifying reading values). The IDE's serial

		monitor is particularly useful during development: we can print out debug messages or sensor readings to ensure our device is perceiving the environment correctly (for instance, printing out detected speed limit value vs. actual speed vs. threshold calculations). The Arduino IDE is cross-platform and lightweight, which suits our development team environment. In essence, the Arduino IDE is the tool that translates our algorithm designs into real firmware running on the microcontroller. By iteratively coding and testing with this IDE, we ensure the embedded side of the project is functioning as intended.
	Android Studio	Android Studio is the official development environment for Android apps. We use Android Studio to design, develop, and test the mobile application component of our project. This powerful IDE provides features such as an XML layout editor for designing app screens, Java/Kotlin code editor with intelligent code completion, and an emulator to run and test the app on virtual devices. With Android Studio, we can integrate Google Maps APIs (if a map view is needed to show vehicle positions), implement a local database or cloud connectivity for fetching the vehicle data, and create a polished user interface. It also helps manage different Android SDK versions to ensure the app is compatible with the range of Android devices used by our target audience. During development, Android Studio's debugger and logging tools allow us to troubleshoot issues like connectivity failures or incorrect data parsing. Given that our app may need to handle continuous data updates, we'll test scenarios such as receiving multiple alerts in quick succession, or running in the background and pushing notifications – Android Studio's profiling tools can assist in optimizing performance and battery usage in such cases. In summary, Android Studio is the all-in-one platform that enables the creation of a robust and user-friendly Android app, which is an integral part of delivering the system's functionality to end users.

	Java (and/or Kotlin) Programming	Java is a primary programming language used for Android app development (alongside Kotlin, which is also common in modern Android development). In our project, we use Java to implement the logic of the Android application – this includes handling network operations (like contacting the cloud server API to retrieve or post data), responding to user inputs, and managing the app's state. For example, when the app receives a JSON response from the server with the latest alerts, Java code will parse that JSON (using libraries like org.json or Google's GSON) into human-readable information that can be displayed. We also rely on Java's threads or Android's AsyncTask (or newer WorkManager/LiveData concepts) to ensure that network operations don't freeze the user interface – maintaining a smooth experience. On the embedded side, while the Arduino code is C++ based, the structure is simple enough to sometimes refer to it as Arduino "sketch" logic rather than full-fledged object-oriented code. However, for the Android side, Java's object-oriented features help us keep the code organized (for instance, defining a class for "VehicleAlert" objects). Java is known for its portability and has been the traditional backbone of Android apps, so using it ensures we have plenty of documentation and community support to draw on. The Android app may also make use of XML for layouts and the manifest, and possibly Kotlin code if we choose some of the modern libraries – but the bulk of the app's functionality (networking, data handling, user interactions) will be implemented in Java. In essence, Java programming is what brings our Android app to life, allowing all the back-end data from the IoT device to be transformed into a meaningful, interactive experience for the user.

4. PROJECT REQUIREMENTS

4.1 Functional Requirements

[1]. **Real-Time Signal Reception and Alert Transmission:**

The system must continuously receive input signals from all sensors (camera, speed sensor, weather sensors) and process them without significant delay. Upon detecting a condition that requires attention (e.g., overspeed, hazardous weather, obstacle), it should transmit an immediate notification. In practice, this means:

- The device should detect a trigger event (like overspeeding) and activate the driver alert within a fraction of a second to a few seconds at most ijraset.com. Stakeholders (e.g., the central monitoring app or nearby vehicles) should receive the alert ideally within a couple of seconds of the event detection (accounting for cellular network latency). Timeliness is crucial – for example, if an elephant or hazard (in the broader context of saving lives, as originally mentioned) is detected, the system should text or notify relevant parties almost instantly.

- The notifications to stakeholders (via GSM or VANET) should contain essential information: vehicle identifier, location, and nature of the alert. This allows recipients to quickly grasp the situation and act researchgate.net. For instance, a fleet manager gets a message "Bus 12: Overspeed 80km/h in 60 zone at GPS [x,y]."

- The system should manage simultaneous conditions gracefully. If multiple alerts trigger quick succession, it should queue or handle them such that all notifications are sent (not lost). The design should consider scenarios like an overspeed alert followed immediately by a sharp braking or accident alert.

[2]. Continuous Monitoring of Vehicle Movements and Road Environment:

The system must continuously monitor the vehicle's speed and surrounding conditions (road signs and weather). This implies:

- It tracks the vehicle's speed in real-time and logs any significant changes. For example, if the vehicle accelerates rapidly or exceeds the speed limit, the system notes this. If the vehicle slows down or stops (like at a red light), the system resets any active overspeed alert.
- It monitors road signs using the camera. Whenever a new sign (speed limit or warning sign) comes into view, the system should recognize and update the current context (e.g., new speed limit zone).

- It monitors weather conditions constantly via sensors or periodic data fetches. If conditions change (rain starts or stops, daylight turns to dusk), the system adapts its behavior (like adjusting the safe speed threshold or switching on headlights if that was a feature).
- Essentially, there should be *no significant downtime* in monitoring; whether the vehicle is moving or stationary, day or night, the sensors and algorithms remain active and ready to respond.

[3]. Integration and Interoperability of System Components:

The system must seamlessly integrate its various components – the Arduino board, sensors (camera, GPS, etc.), communication module, and the mobile app/cloud – to work as a unified whole. This requires:

- Smooth communication between hardware components: e.g., the camera module's output must be correctly interpreted by the Arduino; the GSM module must reliably send messages when commanded; the GPS must feed location data in a compatible format.

- The data exchange format between the device and the cloud/app should be well-defined (like a specific JSON structure or SMS format). This ensures that the mobile application or central server can parse the incoming data without errors.

- If multiple devices (vehicles) are on the network, they should not interfere with each other's communications (for example, if using VANET broadcast, include vehicle ID so devices don't confuse alerts from others as their own).

- The system should support adding additional sensors or modules with minimal reconfiguration. For instance, if in the future we add a front collision radar, the architecture should allow plugging it in and extending the decision logic without a complete redesign. This is more of a forward-looking requirement to ensure scalability and maintainability.

[4]. Dynamic Alert Mechanism (Activation and Deactivation):

The system must manage its alerts in a dynamic fashion – turning them on when a risk is present and off when it subsides, so as not to desensitize or unnecessarily distract the driver.

- **Activation:** Upon detecting an unsafe condition (like the vehicle is too close to an elephant in the original scenario, or overspeed in ours), the system should immediately activate the appropriate alert signals (visual, auditory). If multiple alerts are needed (for different issues), the system should prioritize or indicate accordingly (perhaps a hierarchy: collision imminent > overspeed > mild caution).

- **Deactivation:** Once the condition is resolved (the elephant moved away, or the driver slowed down to below the limit, etc.), the system must promptly deactivate the alertsijraset.com. For example, if the overspeed alert is sounding and the driver reduces speed to within limit, the alert should stop within a second or two. This prevents annoyance and reinforces that the driver's corrective action was recognized.

- The system should avoid "flapping" (rapid on-off-on-off) of alerts, which can happen if a parameter hovers around the threshold. Some hysteresis or time-based smoothing should be implemented. For instance, after the driver corrects speed, perhaps require speed to stay under limit for 3 continuous seconds before silencing the alert, to ensure it wasn't just a momentary dip.

- Additionally, the system should be able to escalate alerts if the condition worsens. For example, a gentle alert if slightly over limit, and a more urgent one if greatly over or if the overspeed persists too long – this encourages compliance.

[5]. Database Logging and Management:

The system must maintain a comprehensive log of significant events and data for future analysis and reference. This translates to:

- Recording every alert event (time, location, vehicle speed vs. limit, weather status, how long until resolved). This data might be stored on the device temporarily and then transmitted to the cloud for persistence.

- Recording normal operational data at intervals (maybe every few minutes log the current speed and location, even if no alert – this can provide context or proof of proper driving).

- The database should handle multiple vehicles feeding data concurrently if deployed in a fleet scenario. Each entry should be tagged with vehicle ID and maybe driver ID (if known) to allow filtering.

- Stakeholders should be able to retrieve and analyze this data. For example, a safety manager could query how many alerts were triggered by bus #5 this month, or what was the average response time of drivers to overspeed alerts.

- The requirement ensures data integrity and availability. Even if the device goes offline (out of GSM coverage), it should queue data and upload when back online so that minimal information is lost. The data is not just for punitive measures but for *insight*:

4.1 Non- Functional Requirements

[1]. Performance

The system is required to operate with high efficiency and responsiveness. This means:

- **Low Latency:** The time from a risk condition occurrence to driver alert should be minimal (preferably under one second for critical events). The processing loop on the Arduino should be optimized to handle sensor inputs swiftly – for example, the image processing for sign recognition should ideally run in real-time (e.g., at least few frames per second so signs aren't missed at driving speed). Notifications to stakeholders should likewise be transmitted almost instantly; overall network delays might vary, but our system should introduce negligible additional delay.

- **Resource Usage:** Despite running on embedded hardware, the system should manage CPU, memory, and network usage efficiently. For instance, the code should avoid heavy computations that aren't necessary or could be done in a simpler way. If using battery or vehicle power, it should be mindful not to over-consume – though in a vehicle, power is less of a problem except if the engine is off.

- **Throughput:** If the system is logging to a database, it should handle the throughput of data appropriately so that it doesn't slow down. This might involve buffering data and sending in batches to not stall the main loop.

- Essentially, performance touches on both speed and efficient operation. The system's high performance ensures it can prevent accidents (a slow warning might be useless) and that it doesn't become a bottleneck or point of failure.

[2]. Reliability

The system must be dependable under various conditions and over extended use. Reliability aspects include:

- **Accuracy:** The detection algorithms should be accurate. False negatives (failing to warn when there is danger) are especially critical to minimize, as they represent missed opportunities to prevent an accident. False positives (warning when no real issue) should also be minimized, as they can annoy and erode trust pmc.ncbi.nlm.nih.gov. For example, the camera should correctly identify the speed limit sign most of the time and not confuse a billboard or another sign as a speed limit. The rain sensor should not trigger falsely due to condensation or a splash unless it truly indicates wet conditions.
- **Robustness:** The system should handle adverse conditions – like low light (use infrared or image adjustments at night for sign detection), extreme heat or cold (the hardware

should function within automotive temperature ranges), vibrations (secure mounting so sensors don't come loose), and electromagnetic interference (proper shielding and filtering for the electronics, especially the communication module).

- **Fault Tolerance:** If a particular sensor fails or gives an implausible reading, the system should detect it and either use a fallback or notify about sensor error, rather than silently failing or giving wrong alerts. For instance, if the camera feed is lost, perhaps default to GPS map speed data if available; if the GSM network is down, still alert the driver and log data to send later.

- **Uptime:** The system should be designed to run whenever the vehicle is in operation (and possibly a bit before/after). It should not crash or hang; memory leaks or bugs that could cause it to freeze over time must be avoided. Watchdog timers might be used to auto-reset in case of any unexpected hang.

- In summary, the reliability requirement ensures the system can be trusted to function and provide correct guidance consistently, which is critical for safety applications.

[3]. Scalability

While our initial deployment might be on a single vehicle, the system should scale well to multiple units and larger contexts.

- The architecture should support adding more vehicles (each with its device) without significant redesign. For example, the communication protocol (IDs etc.) should already accommodate distinguishing multiple vehicles.
- The data backend (database, server, app) should be able to handle simultaneous inputs from numerous vehicles. If 100 buses send updates every minute, the system should cope. This might influence the choice of server and how data is handled (maybe using cloud services that auto-scale).
- Adding additional modules or sensors should be possible. For instance, if later we incorporate a camera for driver drowsiness detection, the system's modular design should allow plugging that in.
- Geographic scalability: If the project expands to nationwide use, the design (especially the comms and data handling) should be suitable. Perhaps consider using more efficient data protocols if SMS becomes a bottleneck (like MQTT over cellular data).
- Essentially, scalability ensures that the system design is not a dead-end good only for a small demo, but could be extended to a more widespread solution, which is important if it proves effective and there's interest in adoption by fleets or authorities.

[4]. Security

Given the system deals with safety-critical commands and sensitive data (vehicle locations, possibly driver behavior data), security is paramount.

- **Data Security:** Any transmission of data (alerts, logs) over the network should be secured to prevent eavesdropping or tampering. For instance, if using a cloud API, use HTTPS encryption. If SMS is used, know that it's not encrypted – perhaps not sending extremely sensitive info via SMS or having a parallel secure channel for aggregated data.

- **Access Control:** The mobile application and database should have proper authentication. Only authorized users should be able to view or query the data. For instance, a user must log in to the app to see vehicles' statuses. Different roles could be considered (admin vs. viewer).

- **Tamper Resistance:** The device hardware should be somewhat tamper-resistant or at least tamper-evident. Because if an ill-intentioned driver tries to disable it (to avoid being monitored), there should be at least a record or an alert of device disconnection. For example, if power is cut, the server might see the vehicle went offline unexpectedly and could flag that.

- **Software Safety:** Ensure the system is safe from malicious inputs – e.g., if receiving data via GSM or VANET from outside, validate it so that bogus or malformed data doesn't crash the system. For VANET, also think about trust: one rogue device shouldn't be able to flood others with false alarms. Some authentication mechanism in V2V messages might be beyond current scope but is a future consideration.

- Maintaining security protects both the system's operation (someone could cause havoc if they spoof alerts or disable them) and privacy of those involved.

[5]. Usability

Finally, for the system to be effective, it has to be user-friendly and intuitive for all human users (drivers, fleet managers, etc.).

- **Driver Interface Usability:** The in-vehicle alerts should be clear and easy to understand. Ideally, the driver doesn't need any special training to interpret them – a red flashing light and "slow down" audible cue is self-explanatory. The alert should also not be overly jarring; it should catch attention but not startle the driver into a panic. Also, it should not be so frequent or unnecessarily triggered that the driver becomes annoyed or turns it off.

- **Mobile App Usability:** The application should present information in a clear and organized way. Key alerts or problem areas should be easily visible (maybe a list of active alerts). The UI should be designed with the workflow of a fleet manager in mind: perhaps a dashboard showing all vehicles and highlighting any that are currently in alert state. Historical data should be accessible through intuitive menus (e.g., selecting a date range or a particular vehicle).

- **Installation and Maintenance:** Usability extends to the people who will install or maintain the system. It should be reasonably plug-and-play in a vehicle, with minimal calibration required. If it needs calibration (like aligning the camera), the instructions should be straightforward. Maintenance like updating the software or checking sensor health should be as simple as possible, maybe via a USB update or OTA update if supported.

- **Documentation and Training:** Provide documentation (or in-app help) for users to understand how the system works, what the alerts mean, and how to respond. For example, instruct drivers that a certain tone means heavy rain caution vs. another tone for overspeed – though ideally different contexts have distinct signals or even voice prompts to avoid confusion.

- Overall, focusing on usability ensures the system will actually be used correctly and not ignored or misinterpreted. A well-designed system that is easy to use is more likely to achieve its safety goals.

5. WORK BREAKDOWN CHART

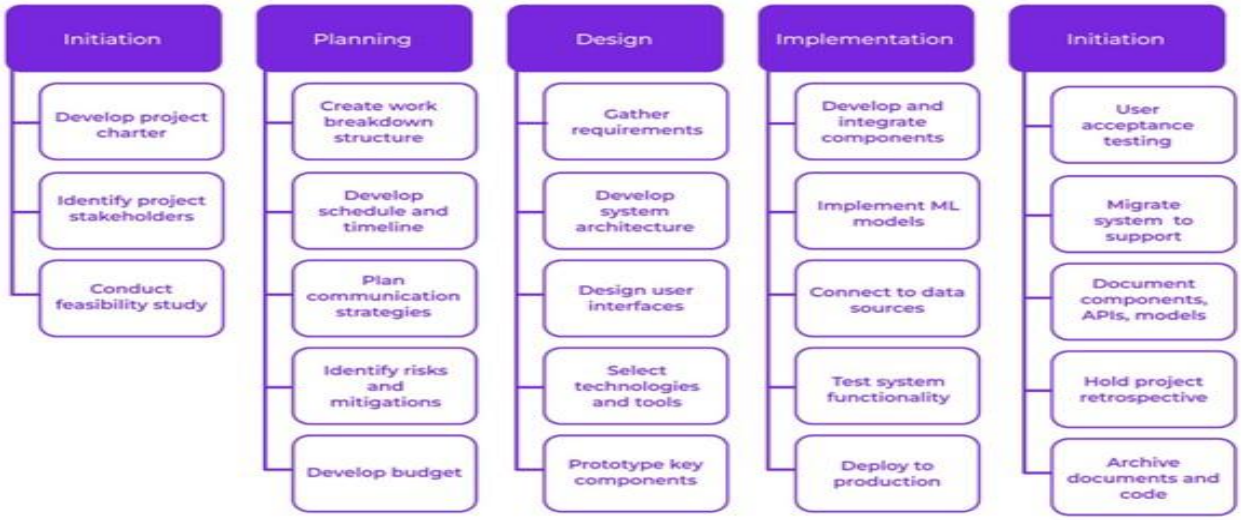


Figure 4: Work Breakdown Structure (WBS) of the Project.

The work breakdown chart (Figure 4) illustrates the decomposition of the project into manageable tasks and sub-tasks. It organizes the project activities into a hierarchical structure to ensure that each aspect of the project is accounted for and can be tracked. At the highest level, the project is divided into several major phases:

- **Initiation & Planning:** This includes early steps such as project discussion workshops, topic selection, and proposal preparation. Each of these has sub-tasks like selecting a supervisor, submitting a project charter, etc.
- **Research & Design:** In this phase, tasks involve conducting the literature survey, identifying the research gap, defining system requirements, and designing the system architecture. Sub-components could include research on sign recognition algorithms, evaluating hardware options, and designing the database schema.
- **Development:** The development phase is broken down into components like hardware setup, software coding for the Arduino, development of the Android app, and integration of the system. Sub-tasks may be prototyping sensor interfacing, implementing the detection algorithm, developing the GSM communication code, etc.
- **Testing:** This covers unit testing of individual modules (camera recognition testing, SMS sending test, etc.), integration testing where all parts are tested together, and user acceptance testing or pilot runs, perhaps in a real vehicle scenario.
- **Project Management & Documentation:** Running parallel to technical tasks, these include preparing progress presentations, writing the research paper, maintaining logbooks, budget management, and final report documentation.

- **Deployment & Evaluation:** Towards the end, tasks like deploying the system for field testing, collecting results, evaluating performance against objectives, and making any necessary refinements are included.
- **Closure:** This entails final presentation (viva), final documentation submission, and wrapping up administrative tasks.

Each of these major blocks is broken into the specific deliverables and tasks as seen in the WBS figure. The hierarchical numbering (for instance, 1.0 for Introduction, 1.1 for Background research, 1.2 for Literature Review, etc., up to tasks like 3.3 for Individual Component development, etc.) corresponds with detailed activities listed in our schedule documents.

By mapping out the tasks in this structured way, the WBS helps the project team monitor progress and ensures nothing essential is overlooked. It also facilitates delegation of tasks among team members, timeline estimations, and recognizing dependencies between tasks (for example, hardware development might need to precede full integration testing). The work breakdown chart is essentially the project's blueprint from a task perspective, guiding the team through to project completion.

6. GANTT CHART

Table 3: Gantt Chart

7. BUDGET AND BUDGET JUSTIFICATION

Determining the budget for the development and implementation of the Elephant Detection System for Railway Lines requires a comprehensive analysis of various factors, including the scope of the project, technology requirements, personnel costs, infrastructure expenses, and ongoing operational expenses.

While providing an exact budget without detailed information is challenging, I can outline some key cost considerations

Table 4: Budget and Budget Justification

Component	Amount
ESP32-CAM module	Rs. 1,700
ESP32DevKit (WROOM)	Rs.2,800
GPS NEO-6M	Rs. 3,500
Rain sensor	Rs. 750
LDR (light sensor)	Rs.300
DHT22 (temp/humidity)	Rs. 1,500
Buzzer + high-bright LEDs	Rs. 1,200
DC-DC buck 12V→5V (3A) + fuses	Rs.1800
Level shifter / SIM power regulator	Rs. 800
Antennas + SIM activation/credit	Rs. 1,000
Prototype wiring/headers/PCB	Rs. 2,000
MicroSD 16 GB	Rs.1800
IP54 enclosure + mounts	Rs.3000
Camera mount bracket	Rs.800
TFT 2.4″ SPI display	Rs.3500
Backup Li-Ion + charger (UPS)	Rs.3000
Shielded cables / rugged connectors	Rs. 1,500
Small PCB proto run	Rs. 2,500
Total	Rs. 33,450

8. REFERENCES

[1] FHWA Road Weather Management Program. “How Do Weather Events Affect Roads?” (accessed Jul 2025). FHWA Operations

[2] FHWA Road Weather Management Program. “Rain & Flooding” statistics page (updated Sep 2024). FHWA Operations

[3 Aarts, L., & van Schagen, I. (2006). *Driving speed and the risk of road crashes: A review. Accident Analysis & Prevention, 38*(2), 215–224. PubMedدانشیاری | دانستنی‌های جذاب برای زندگی

[4] Elvik, R. (2019). *Updated estimates of the relationship between speed and road safety. Accident Analysis & Prevention, 123*, 114–122. (Abstract) ScienceDirect

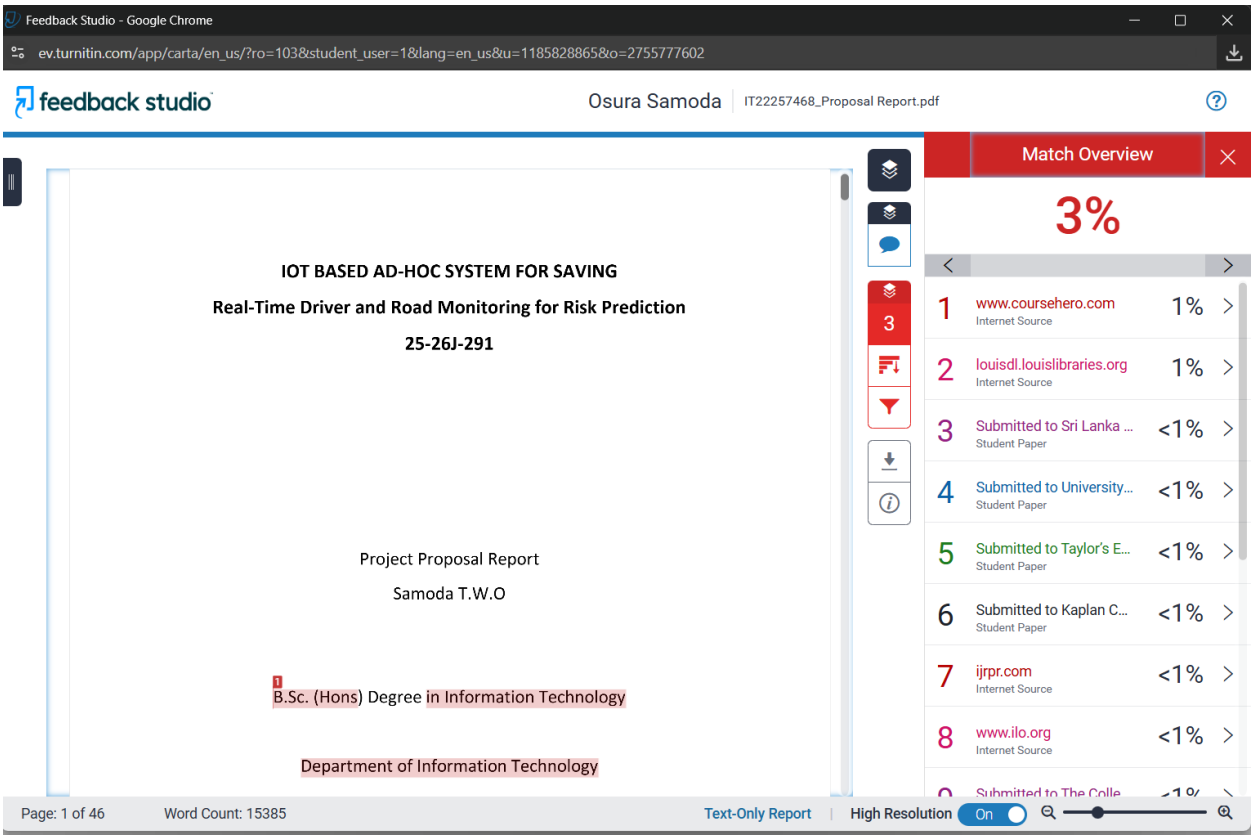
[5] Zanella, A., et al. (2014). *Internet of Things for Smart Cities*. **IEEE Internet of Things Journal**, 1(1), 22-32. [Discusses IoT applications in smart city infrastructure; notes how IoT can enable smart road systems that communicate safety data to driversijraset.com.]

[6] Pushpakumara, T. D. C., Senanayake, I. P., & Puswewala, U. G. A. (2012). *Impact of Rain on Road Accidents in Sri Lanka*. **Global Journal of Engineering & Applied Sciences**, 2(4), 297-300. [Establishes correlation between rainfall and increased road accident rates in Colombo; emphasizes that weather conditions significantly affect road safetyresearchgate.net.]

[7 Pushpakumara, T.D.C., Senanayake, I.P., & Puswewala, U.G.A. (2012). *Impact of Rain on Road Accidents in Sri Lanka. Global Journal of Engineering & Applied Sciences, 2*(4), 297–300. rrjponline.com

[8] Zanella, A., Bui, N., Castellani, A., Vangelista, L., & Zorzi, M. (2014). *Internet of Things for Smart Cities. IEEE Internet of Things Journal, 1*(1), 22–32. SCIRPAcademia

[9] European Commission. “Intelligent Speed Adaptation (ISA)” – European Road Safety Observatory explainer. Mobility & Transport - Road Safety

[10] Murphy, R. et al. (2012). *Best Practices for Road Weather Management.* FHWA-HOP-12-046. transops.s3.amazonaws.com

Turtnitin Plagiarism check